

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3

По дисциплине

“Основы программной инженерии”

Вариант: 735822

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р3413

Преподаватель:
Обляшевский Севастьян Владиславович

Санкт-Петербург, 2025 г

Текст задания

Написать сценарий для утилиты Apache Ant, реализующий компиляцию, тестирование и упаковку в jar-архив кода проекта из лабораторной работы №3 по дисциплине "Веб-программирование".

Каждый этап должен быть выделен в отдельный блок сценария; все переменные и константы, используемые в сценарии, должны быть вынесены в отдельный файл параметров; MANIFEST.MF должен содержать информацию о версии и о запуске класса.

Сценарий должен реализовывать следующие цели (targets):

1. **compile** -- компиляция исходных кодов проекта.
2. **build** -- компиляция исходных кодов проекта и их упаковка в исполняемый jar-архив. Компиляцию исходных кодов реализовать посредством вызова цели **compile**.
3. **clean** -- удаление скомпилированных классов проекта и всех временных файлов (если они есть).
4. **test** -- запуск junit-тестов проекта. Перед запуском тестов необходимо осуществить сборку проекта (цель **build**).
5. **xml** - валидация всех xml-файлов в проекте.
6. **scp** - перемещение собранного проекта по scp на выбранный сервер по завершению сборки. Предварительно необходимо выполнить сборку проекта (цель **build**).
7. **native2ascii** - преобразование native2ascii для копий файлов локализации (для тестирования сценария все строковые параметры необходимо вынести из классов в файлы локализации).
8. **music** - воспроизведение музыки по завершению сборки (цель **build**).
9. **doc** - добавление в MANIFEST.MF MD5 и SHA-1 файлов проекта, а также генерация и добавление в архив javadoc по всем классам проекта.
10. **env** - осуществляет сборку и запуск программы в альтернативных окружениях; окружение задается версией java и набором аргументов виртуальной машины в файле параметров.
11. **alt** - создаёт альтернативную версию программы с измененными именами переменных и классов (используя задание replace/replaceRegExp в файлах параметров) и упаковывает её в jar-архив. Для создания jar-архива использует цель **build**.
12. **diff** - осуществляет проверку состояния рабочей копии, и, если изменения не касаются классов, указанных в файле параметров выполняет commit в репозиторий git.
13. **report** - в случае успешного прохождения тестов сохраняет отчет junit в формате xml, добавляет его в репозиторий git и выполняет commit.
14. **history** - если проект не удаётся скомпилировать (цель **compile**), загружается предыдущая версия из репозитория svn. Операция повторяется до тех пор, пока проект не удастся собрать, либо не будет получена самая первая ревизия из репозитория. Если такая ревизия найдена, то формируется файл, содержащий результат операции diff для всех файлов, изменённых в ревизии, следующей непосредственно за последней работающей.
15. **team** - осуществляет получение из svn-репозитория 2 предыдущих ревизий, их сборку (по аналогии с основной) и упаковку получившихся jar-файлов в zip-архив. Сборку реализовать посредством вызова цели **build**.

Ход выполнения

Исходный код 3 лабораторной работы по дисциплине «Веб-программирование» - <https://github.com/stoneshik/third-lab-web>

Исходный код данной лабораторной (измененная лабораторная по «Веб-программированию», с реализованным сценарием Apache Ant) — <https://github.com/stoneshik/opi-third-lab>

Скачиваем зависимости maven в директорию /lib:
mvn dependency:copy-dependencies -DoutputDirectory=lib

Для работы с Junit 5 через Apache Ant нужен standalone runner, скачиваем его:
wget https://repo1.maven.org/maven2/org/junit/platform/junit-platform-console-standalone/1.8.2/junit-platform-console-standalone-1.8.2.jar -O lib/junit-platform-console-standalone.jar

Поскольку изначальное приложение JSF — в нем нет Main-Class, поэтому для того чтобы бы собирался исполняемый JAR создаем файл src/main/java/lab/third/MainClass.java

Также добавляем Junit тест для **test** и JavaDoc'и для **doc**.

Для **scp** используем локальную директорию /tmp/demo_scp

Также сгенерируем ssh ключ:

```
ssh-keygen -t rsa -b 4096 -m PEM -f ~/.ssh/id_rsa_ant -N ""
```

Копируем публичный ключ на сервер (localhost):

```
cat ~/.ssh/id_rsa_ant.pub >> ~/.ssh/authorized_keys
```

```
chmod 600 ~/.ssh/authorized_keys
```

```
chmod 700 ~/.ssh
```

Проверка подключения:

```
ssh -i ~/.ssh/id_rsa_ant sus@localhost
```

Старт ssh localhost:

```
sudo systemctl start ssh
```

Добавляем файл локализации src/main/resources/i18n-src/messages_ru.properties для **native2ascii**.

Добавляем музыкальный файл resources/music/complete.mp3 для **music**.

Создаем файлы **build.properties** и **build.xml** — первый содержит все параметры/константы для сценария Apache Ant, второй реализует сам сценарий.

build.properties: <https://github.com/stoneshik/opi-third-lab/blob/main/build.properties>

build.xml: <https://github.com/stoneshik/opi-third-lab/blob/main/build.xml>

Для **history** и **team** требуется работа с svn репозиторием, для демонстрации будем использовать локальный bare репозиторий, который будет создаваться при помощи следующего скрипта **svn.sh**:

```
#!/bin/bash
set -e
BASE_DIR="$HOME/svn_demo"
REPO_DIR="$BASE_DIR/repo"
WORK_TRUNK="$BASE_DIR/work_trunk"
WORK_SECOND="$BASE_DIR/work_second"
PROJECT_DIR="$HOME/Desktop/university/opi/third/third-lab-web-master"

echo "=== Очистка старых данных ==="
rm -rf "$BASE_DIR"
mkdir -p "$BASE_DIR"

echo "=== Создание SVN репозитория ==="
svnadmin create "$REPO_DIR"
REPO_URL="file://$REPO_DIR"

svn mkdir "$REPO_URL/trunk" "$REPO_URL/branches" "$REPO_URL/tags" -m "Initial structure"

echo "=== Чекаут trunk ==="
svn checkout "$REPO_URL/trunk" "$WORK_TRUNK"

# r0: initial commit
cp -r "$PROJECT_DIR/*" "$WORK_TRUNK/"
cd "$WORK_TRUNK"
svn add * --force
svn commit -m "r0: Initial commit"

# r1: небольшое изменение
echo "// demo change r1" >> src/main/java/lab/third/util/TransactionExecutor.java
svn commit -m "r1: Demo update in trunk"

# r2: создает ветку second
svn copy "$REPO_URL/trunk" "$REPO_URL/branches/second" -m "r2: Create branch second"

# r3: изменение в ветке second
svn checkout "$REPO_URL/branches/second" "$WORK_SECOND"
cd "$WORK_SECOND"
echo "// demo change r3 in second" >> src/main/java/lab/third/beans/DotManagedBean.java
svn commit -m "r3: Demo update in branch second"

echo "=== Локальный SVN репозиторий готов для демонстрации ==="
echo "Репозиторий: $REPO_DIR"
echo "Рабочие копии: trunk=$WORK_TRUNK, second=$WORK_SECOND"
echo "URL репозитория: $REPO_URL"
```

В **alt** (11 target) — используется скрипт на python для исправления названия файла, если название класса было изменено — https://github.com/stoneshik/opi-third-lab/blob/main/scripts/rename_classes.py

В **diff** (12 target) — используется скрипт на python для проверки есть ли среди измененных файлов запрещенные — https://github.com/stoneshik/opi-third-lab/blob/main/scripts/git_diff_check.py

В **history** (14 target) — используется скрипт на python который выполняет всю логику цели (пытается собрать предыдущие SVN ревизии пока не соберется проект) — <https://github.com/stoneshik/opi-third-lab/blob/main/scripts/history.py>

В **team** (15 target) — используется скрипт на python который выполняет всю логику цели (получает 2 предыдущие ревизии из svn, собирает и упаковывает jar-файлы в zip) — <https://github.com/stoneshik/opi-third-lab/blob/main/scripts/team.py>

Выводы по работе

В ходе выполнения лабораторной работы был разработан сценарий сборки на Apache Ant для автоматизации компиляции, тестирования и упаковки проекта в jar-архив. Все параметры сборки вынесены в файл build.properties, а процесс разделён на отдельные цели, соответствующие этапам жизненного цикла проекта.

В рамках работы были реализованы сборка и запуск тестов JUnit, валидация XML, генерация документации и контрольных сумм, альтернативные сборки и окружения, а также интеграция с системами контроля версий Git и SVN.

В результате были получены практические навыки настройки Apache Ant и автоматизации сборки Java-проектов